

GIT MASTERY GUIDE

Scenario-Based Learning for DevOps & MLOps Engineers

01 Git Configuration & Setup

03 Branching Workflows

05 Stashing & Recovery

07 Tags & Versioning

02 Staging Area & Commits

04 Fetch, Pull & Rebase

06 Reflog — Git's Safety Net

08 Complex Scenarios & Quick Reference

Chapter 1

Git Configuration

Before writing a single commit, Git needs to know who you are and how you like to work. These global settings follow you across every repository on your machine.

1.1 Setting Your Editor

Scenario

You run `git commit` and a strange terminal UI appears — Vim — with no obvious way to type or exit. You use VS Code every day. Here is how to fix it once and for all.

Git needs a text editor whenever it requires your input — writing commit messages, interactive rebases, merge confirmations. By default it opens Vim. The fix is:

```
# Set VS Code as your Git editor (recommended)
git config --global core.editor "code --wait"

# Or set Vim if you prefer terminal editors
git config --global core.editor vim
```

The `--wait` flag is critical for VS Code. Without it, Git opens the editor and immediately reads the still-empty file, producing an empty commit message. With `--wait`, Git pauses until you close the tab, then reads your message.

Tip

Your code editor and Git's editor are completely separate things. This setting only affects moments when Git needs to ask you something — not where you write your code.

Warning

Changing this setting has zero effect on existing repositories, commit history, or any of your files. It is purely a preference setting stored in `~/.gitconfig`.

1.2 Pull Strategy — Rebase vs Merge

Scenario

You run `git pull` and Git creates a mysterious extra merge commit even though you only downloaded someone else's changes. Your history looks tangled. Set this once to keep history clean forever.

```
# Always rebase on pull (recommended for clean history)
git config --global pull.rebase true

# Check your current setting
git config --global pull.rebase

# true = rebase | false = merge | blank = merge (default)
```

Strategy	What it does	History shape
Merge (default)	Creates an extra merge commit joining both lines	Branchy, harder to read over time
Rebase (recommended)	Replays your commits on top of remote	Clean and linear

1.3 Default Branch Name

Scenario

You create a local repo with `git init`, then push to GitHub. Git says branches don't match — your local branch is `master` but GitHub uses `main`.

```
git config --global init.defaultBranch main

# Verify
git config --global init.defaultBranch
```

This only affects new repositories you create in the future. Existing repos are completely untouched.

Command	Purpose
<code>git config --global core.editor "code --wait"</code>	Set VS Code as Git editor
<code>git config --global pull.rebase true</code>	Rebase instead of merge on pull
<code>git config --global pull.rebase false</code>	Merge on pull (default)
<code>git config --global init.defaultBranch main</code>	Default branch name for new repos
<code>git config --global pull.rebase</code>	Check current pull strategy

Chapter 2

Staging Area & Commits

The staging area is the key to writing clean, intentional commit history. Understanding it separates beginners from professionals.

2.1 The Three Zones of Git

Working Directory → Staging Area → Repository

(where you code) (what you (permanent
prepare) history)

Working Directory — where you freely edit files. **Staging Area** — a deliberate selection of changes for the next commit. **Repository** — permanent, immutable commit history.

2.2 Why the Staging Area Exists

Scenario

You spent the afternoon fixing a bug AND refactoring unrelated code. You want two separate commits so your history stays meaningful. The staging area makes this possible.

```
# You edited: bugfix.py, refactor.py, notes.txt

git add bugfix.py

git commit -m 'fix: resolve null pointer in inference pipeline'

git add refactor.py

git commit -m 'refactor: simplify data loader logic'

# notes.txt never staged — stays in working directory
```

Tip

Think of staging like packing a shipping box. ``git add`` puts items in the box. ``git commit`` seals and ships it. You decide exactly what goes in.

2.3 Fixing Your Last Commit — `git commit --amend`

Scenario

You just committed and immediately noticed a typo in the commit message. Or you forgot to include one file. Do NOT make a new commit — use amend.

```
# Fix only the commit message
git commit --amend -m 'corrected message'

# Add a forgotten file, keep the same message
git add forgotten_file.py
git commit --amend --no-edit

# Fix both the file AND the message at once
git add forgotten_file.py
git commit --amend -m 'feat: add batch inference endpoint'
```

Warning

Only amend commits you have NOT pushed to remote. Amending a pushed commit rewrites history and causes conflicts for your collaborators.

2.4 Pushing Multiple Local Commits

Scenario

You committed a change to model.py, then modified the same file again. Now you want to push both changes. What is the best approach?

```
# Simply make the second commit normally and push
git add .
git commit -m 'feat: second improvement to model'
git push origin main

# Git automatically sends BOTH commits in one push ☑
```

You can accumulate as many local commits as you like before pushing. Git tracks exactly which commits the remote has and sends only what's missing. This is perfectly normal and expected workflow.

Chapter 3

Branching Workflows

Two professional strategies — Trunk-Based Development for continuous delivery and GitFlow for explicit versioned releases.

3.1 Trunk-Based Development

The core principle: feature branches live for **hours, not weeks**. Everyone integrates to main at least daily. CI stays relevant, main stays deployable.

```
# Step 1 – Always start from the latest main
git checkout main && git pull

# Step 2 – Create a short-lived feature branch
git checkout -b feat/batch-inference

# Step 3 – Make small focused changes, stage deliberately
git add -p # stage chunk by chunk
git commit -m 'feat: add batch inference endpoint'

# Step 4 – Sync with main before opening PR
git fetch origin
git rebase origin/main

# Step 5 – Resolve conflicts if any
git status

# edit file: fix <<<<<< ===== >>>>>> markers
git add src/serve.py
git rebase --continue

# Step 6 – Push feature branch and open PR
git push origin feat/batch-inference
```

Scenario

A collaborator pushed commits C and D to the shared `temp` branch while you were working locally on E and F. Running `git push` is rejected with 'non-fast-forward'.

```
# CORRECT – never force push on a shared branch
```

```
git fetch origin
git rebase origin/temp
# resolve any conflicts, then:
git push origin temp
```

Warning

Never use `git push --force` on a shared branch. It overwrites your collaborators' commits and destroys their work.

3.2 Branch Cleanup

Scenario

After months of work your machine has dozens of stale branches. Every branch you list adds noise. Here is how to keep things tidy.

```
# Delete locally after PR is merged
git branch -d feat/batch-inference

# Delete on remote (GitHub also shows a Delete button after merge)
git push origin --delete feat/batch-inference

# Remove local references to branches already deleted on remote
git fetch --prune

# See all branches
git branch -a
```

Tip

Branches are meant to be temporary and disposable. Create freely, merge quickly, delete immediately. At any point you should only have a handful of active branches — one per feature in progress.

3.3 GitFlow — For Versioned Releases

GitFlow is ideal when you ship explicit versioned releases like v1.0, v2.0 — common in ML model deployment and software products. It is built on top of regular Git, so you can mix git and git-flow commands freely.

- **main** — Production-ready code only. Always tagged with a version number.
- **develop** — Integration branch. All features merge here first.
- **feature/*** — New functionality. Born from develop, merged back to develop.

- [release/*](#) — Release preparation: version bump, changelog, final fixes.
- [hotfix/*](#) — Emergency fixes applied directly to main, then backported to develop.

```
git flow init

# Feature lifecycle
git flow feature start new-model # branch from develop
git flow feature finish new-model # merge back, delete branch

# Release lifecycle
git flow release start 1.2.0 # branch from develop
git flow release finish 1.2.0 # merge to main + develop, tag v1.2.0

# Emergency hotfix
git flow hotfix start fix-oom # branch from main
git flow hotfix finish fix-oom # merge to main + develop, tag
```

Workflow	Branch Life & Best For	Complexity
Trunk-Based	Hours — continuous deployment, web apps	Lower
GitFlow	Days-weeks — versioned releases, ML products	Higher

Chapter 4

Fetch, Pull & Rebase

Understanding these commands is essential for safe collaboration on shared branches.

4.1 git fetch vs git pull

Scenario

You are on a feature branch mid-work and want to see what teammates pushed to remote — without accidentally overwriting your local changes.

```
git fetch origin # downloads remote changes – does NOT touch your files
git pull # downloads AND immediately merges/rebases into your branch
```

Behaviour	git fetch	git pull
Downloads from remote	Yes — safe	Yes
Touches your working files	No	Yes — immediate
You stay in control	Yes	Git decides
Good for feature branches	Yes — preferred	Risky
Good for morning main sync	Needs extra rebase step	Yes — quick

4.2 git pull variations

The behaviour of plain `git pull` depends entirely on your global config:

```
git pull # uses pull.rebase config (true = rebase, false = merge)
git pull --rebase # always rebase, overrides config this one time
git pull --no-rebase # always merge, overrides config this one time
```

git pull --no-rebase creates an extra merge commit even when you just want to download changes. Over time with many collaborators this produces tangled, branchy history. Prefer **git fetch** + **git rebase** on feature branches.

4.3 Interactive Rebase — Rewriting History

Scenario

You made 4 local commits while building a feature. Three are messy fix-up commits. Before pushing you want to squash them into one clean professional commit.

```
git rebase -i HEAD~4 # open last 4 commits in editor

# Editor shows:

pick a1b2c3d feat: add training loop
pick b2c3d4e fix: forgot to save file
pick c3d4e5f fix: another typo
pick d4e5f6g fix: removed debug print

# Change to:

pick a1b2c3d feat: add training loop
squash b2c3d4e fix: forgot to save file
squash c3d4e5f fix: another typo
squash d4e5f6g fix: removed debug print

# Save and close – all 4 become ONE clean commit
```

Editor command	Effect
<code>pick</code>	Keep the commit exactly as is
<code>reword</code>	Keep commit but edit its message
<code>squash</code>	Merge into previous commit, combine messages
<code>fixup</code>	Merge into previous commit, discard this message
<code>drop</code>	Delete this commit entirely

Warning

Never use interactive rebase on commits already pushed to remote. Rebase rewrites commit hashes — collaborators who have those commits will get conflicts.

4.4 Undoing a Rebase

Scenario

You ran ``git rebase origin/main`` and things went badly. You want to go back to exactly where you were before.

```
# Option 1 – Rebase still running
```

```
git rebase --abort

# Option 2 – Rebase just finished (immediate undo)
git reset --hard ORIG_HEAD

# Option 3 – Time has passed, use reflog
git reflog

# find HEAD@{n} from just before the rebase started
git reset --hard HEAD@{4}
```

Command	What it does
<code>git rebase --abort</code>	Cancel rebase while it is still running
<code>git reset --hard ORIG_HEAD</code>	Undo a just-completed rebase immediately
<code>git reflog</code>	Find any past HEAD position to reset to

Chapter 5

Stashing — Temporary Storage

Stash lets you shelve work in progress and restore it later, even on a completely different branch.

5.1 Basic Stash

Scenario

You are halfway through implementing fp16 quantization when a teammate says: 'Fix this urgent bug on main right now.' Your code is half-done — not ready to commit, can't be thrown away. Stash saves it.

```
# Save current changes temporarily with a label
git stash push -m 'WIP: testing fp16 quantization approach'

# Working directory is now clean — switch freely
git checkout main

# ... fix the bug, commit, push ...

# Come back and restore your work
git checkout feat/quantization
git stash pop
```

Stash is **global to the entire repository** — not branch-specific. You can stash on one branch and pop on any other. This is especially useful when you accidentally start coding on the wrong branch.

Tip

The only difference between ``git stash`` and ``git stash push -m 'message'`` is the label. Both do the same thing — the message just makes it easier to identify when you have multiple stashes.

5.2 Managing Multiple Stashes

Scenario

You stashed 2 separate experiments and have uncommitted changes for a third. At the end of the day you want all three combined into one commit.

```
git stash list

# stash@{0}: On main: WIP: changed file2
```

```
# stash@{1}: On main: WIP: changed file1
# file3 changes still sitting in working directory

# Apply both – use 'apply' not 'pop' (keeps stash as backup)
git stash apply stash@{1} # restore file1 changes
git stash apply stash@{0} # restore file2 changes
# file3 already in working directory

# All three combined – commit together
git add .
git commit -m 'feat: combined three experiments'
git stash clear
```

Warning

If two stashes touched the same file, applying them will produce conflict markers just like a rebase conflict. Edit the file to resolve, then `git add` the file to mark it resolved.

5.3 Moving Stashed Work to a New Branch

Scenario

You stashed experimental changes on main, but the work grew too large. It deserves its own feature branch – and you want zero conflicts.

```
git stash branch feat/fp16-quantization

# This single command:
# 1. Goes back to the commit where you originally stashed
# 2. Creates the new branch from that exact point
# 3. Applies your stashed changes cleanly
# 4. Drops the stash automatically
```

The key advantage: by going back to the exact commit where you stashed, changes apply with **zero conflicts** even if main has moved forward since.

Command	What it does
<code>git stash push -m 'msg'</code>	Save with a descriptive label
<code>git stash list</code>	See all saved stashes
<code>git stash apply stash@{n}</code>	Restore but keep in stash list (safe)

Command	What it does
<code>git stash pop</code>	Restore and remove from list
<code>git stash drop stash@{n}</code>	Delete one specific stash
<code>git stash clear</code>	Delete all stashes
<code>git stash branch new-branch</code>	Move stash to a new branch cleanly

Chapter 6

Reflog — Git's Safety Net

Reflog records every movement of HEAD. Nothing is truly lost as long as it is in the reflog.

6.1 What Reflog Is

Reflog is Git's secret diary. Every time HEAD moves — commit, checkout, reset, rebase, merge — Git writes an entry. Even operations that look permanently destructive are recorded here.

```
git reflog

# Example output:

a1b2c3d HEAD@{0}: reset: moving to HEAD~3 <- what we just did
j0k1l2m HEAD@{1}: commit: feat: add model v2
e4f5g6h HEAD@{2}: checkout: switching branches
```

Most recent action is at the top. Each entry shows the commit hash at that moment, the HEAD position index HEAD@{n}, and a description of what caused HEAD to move.

6.2 Recovering Lost Commits

Scenario

You ran `git reset --hard HEAD~3` to undo some work, then immediately regretted it. Those commits look gone — but they are not.

```
git reflog

# a1b2c3d HEAD@{0}: reset: moving to HEAD~3 <- the mistake
# j0k1l2m HEAD@{1}: commit: feat: add model v2
# e4f5g6h HEAD@{2}: commit: feat: add data loader
# f7g8h9i HEAD@{3}: commit: feat: initial model

# Recover — reset to just before the destructive reset
git reset --hard HEAD@{1}

# or use the commit hash directly
git reset --hard j0k1l2m
```

Tip

Reflog entries expire after 90 days. Reflog is local only — it does not exist on GitHub or remote. Use it quickly after an accident.

6.3 git blame — Finding Who Changed What

Scenario

Something broke in `src/model.py`. You need to know who changed line 45 and why, so you can understand the context before fixing it.

```
git blame src/model.py

# Output:
a1b2c3d (John 2024-01-15 10:23) 1: import tensorflow as tf
e5f6g7h (Sarah 2024-01-18 14:10) 2: import numpy as np
k9l0m1n (Mike 2024-02-01 09:45) 4: class Model:

# Blame a specific range of lines only
git blame -L 40,50 src/model.py

# Then inspect the full commit for context
git show k9l0m1n
```

`git blame` is a **detective tool**, not an accusation tool. It helps you find the right person to ask about a piece of code and understand the original intent behind a change.

Chapter 7

Tags & Versioning

Tags are permanent named markers on specific commits — essential for ML model versioning and release management.

7.1 Creating and Pushing Tags

Scenario

Your ML model just reached 92% accuracy on the test set. You want to mark this exact commit permanently so you can always return to this exact version.

```
# Create annotated tags with descriptive messages
git tag -a v1.0.0 -m 'Model v1.0: baseline FP32, 92% accuracy'

git tag -a v1.1.0 -m 'Model v1.1: FP16 inference, 40% latency improvement'

# Tag a specific past commit (if you forgot to tag earlier)
git tag -a v1.0.0 a1b2c3d -m 'Tag a specific past commit'

# Push tags – they do NOT push automatically with git push
git push origin --tags # push ALL tags
git push origin v1.1.0 # push ONE specific tag

# List all tags
git tag
```

Warning

Tags do NOT push automatically with `git push`. You must explicitly push them with `--tags` or by naming the tag.

7.2 Describing the Current Version

Scenario

You are writing a deployment script and need to know exactly what version of the model is currently checked out.

```
git describe --tags --long

# Output: v1.1.0-3-ga1b2c3d
```

```
# ^ ^ ^  
  
# tag | short commit hash  
  
# commits after tag  
  
git describe --tags --abbrev=0  
# Output: v1.1.0 (just the tag name)  
  
# In deployment scripts:  
VERSION=$(git describe --tags --abbrev=0)  
echo "Deploying model version $VERSION"
```

Command	What it does
<code>git tag</code>	List all existing tags
<code>git tag -a v1.0.0 -m 'message'</code>	Create annotated tag on current commit
<code>git tag -a v1.0.0 -m 'msg'</code>	Tag a specific past commit
<code>git push origin --tags</code>	Push ALL tags to remote
<code>git push origin v1.0.0</code>	Push one specific tag
<code>git describe --tags --long</code>	Full version: tag + commit count + hash
<code>git describe --tags --abbrev=0</code>	Just the nearest tag name

Chapter 8

Complex Scenarios & Quick Reference

Real-world situations that combine multiple Git concepts in a single workflow.

8.1 The ML Engineer's Busy Day

Scenario

You are a ML Engineer working on a SAR image classification model. Here is a realistic day that combines stash, branches, hotfixes, and rebasing.

```
# Morning – experimenting with fp16 quantization
# modify model.py ...
git stash push -m 'WIP: fp16 quantization experiment'

# Teammate asks you to review their data pipeline branch
git checkout feat/data-pipeline
# make review notes in dataloader.py ...
git stash push -m 'WIP: data pipeline review notes'

# Urgent bug reported on main
git checkout main
git checkout -b hotfix/sar-image-crash
git add inference.py
git commit -m 'fix: SAR image crash on empty batch'
git push origin hotfix/sar-image-crash

# PR opened, reviewed, merged

# Back to pipeline review
git checkout feat/data-pipeline
git stash pop
# finish review

# Back to fp16 experiment
git checkout main && git pull
git stash apply stash@{0}
git add .
```

```
git commit -m 'feat: fp16 quantization, 40% latency improvement'

git stash clear

git push origin main
```

8.2 Cleaning Up Before a PR

Scenario

You built a feature over 2 days with 8 messy commits. Before opening a PR you want clean, meaningful history.

```
# Step 1 – See what is unpushed
git log origin/main..HEAD --oneline

# Step 2 – Squash the messy commits
git rebase -i HEAD~4

# squash fix commits into the feat commit

# Step 3 – Sync with latest main
git fetch origin
git rebase origin/main

# Step 4 – Push clean history
git push origin feat/batch-inference
```

8.3 Master Quick Reference

Command	What it does
<code>git log --oneline</code>	Clean one-line commit history
<code>git log origin/main..HEAD --oneline</code>	Show only unpushed commits
<code>git log --oneline -5</code>	Last 5 commits
<code>git status</code>	Staged, unstaged, and untracked files
<code>git diff</code>	See unstaged changes
<code>git diff --staged</code>	See staged changes
<code>git fetch origin</code>	Download remote changes without applying
<code>git rebase origin/main</code>	Replay your commits on top of latest main

Command	What it does
<code>git rebase -i HEAD~n</code>	Interactive rebase — rewrite last n commits
<code>git rebase --abort</code>	Cancel an in-progress rebase
<code>git reset --hard ORIG_HEAD</code>	Undo a just-completed rebase
<code>git reset --hard HEAD@{n}</code>	Go to any past HEAD position from reflog
<code>git reflog</code>	Full history of every HEAD movement
<code>git blame</code>	See who wrote each line of a file
<code>git stash push -m 'msg'</code>	Stash work in progress with a label
<code>git stash branch new-branch</code>	Move stash to a new branch, no conflicts
<code>git fetch --prune</code>	Remove stale remote-tracking branches
<code>git pull --rebase</code>	Pull and rebase — overrides config once
<code>git pull --no-rebase</code>	Pull and merge — overrides config once
<code>git describe --tags --abbrev=0</code>	Current version tag name